LongPost v3.2.3 技术架构与产品白皮书

文档定位：面向架构评审、团队交接、技术选型与二次开发者的完整说明。

覆盖范围：Web 云端、桌面工作台、Android、Chrome 扩展、本地桥与生态接入。

数据基准：仓库 `main` 分支 v3.2.3（提交 `dd84c3f`）。

配套文档：产品报告 · 使用说明 · 剪藏 API · 生态接入

目录

一、 执行摘要

1.1 长文创作的三重断裂

1.2 LongPost 的答案

1.3 关键数字（v3.2.3 实测）

二、 版本演进

三、 系统架构总览

3.1 分层

3.2 为什么不做成一套

3.3 一次阅读请求的完整链路

四、 三端形态与存储拓扑

4.1 桌面端存储细节

4.2 移动端

五、 Web 云端平台

5.1 模块地图（21 个）

5.2 路由分布

5.3 SSR 与客户端渲染的边界

5.4 数据库访问

六、 桌面工作台

6.1 进程结构

6.2 为什么渲染层不用框架

6.3 IPC 面（节选）

6.4 打包清单守卫

七、 双工作区设计系统

7.1 两套骨架，不是两套配色

7.2 设计令牌

7.3 响应式的真实边界

八、 内容引擎

8.1 Markdown 渲染

8.2 图表 DSL

8.3 导出矩阵

8.4 排版算法

九、 AI 体系

9.1 三级取用

9.2 网关设计

9.3 密钥托管

9.4 场景化能力

十、 v3.0 三件套

10.1 订阅（Newsletter）

10.2 分发（OG 分享图）

10.3 迁移（导入 / 导出）

十一、 v3.1–3.2 生态层

11.1 本地桥协议

11.2 一个必须讲清楚的坑：Private Network Access

11.3 端口与令牌发现

11.4 Chrome 扩展（MV3）

11.5 命令行与 Obsidian

十二、 安全体系

12.1 威胁模型与处置

12.2 供应链治理

12.3 可选功能不得拖垮启动

十三、 质量工程

13.1 测试矩阵

13.2 三个「守卫」脚本

13.3 CI 流水线

13.4 质量门禁

十四、 性能与产物

十五、 部署与运维

15.1 三种方式

15.2 必填环境变量

15.3 可选：订阅与 AI

15.4 备份

十六、 数据模型

十七、 已知限制与路线图

17.1 限制

17.2 路线图

十八、 附录

18.1 目录结构

18.2 常用命令
18.3 快捷键
18.4 剪藏 API 速查
十九、 关键设计决策记录（ADR）
ADR-01：桌面与 Web 不共享 UI 代码
ADR-02：测试直连生产代码
ADR-03：渲染层不引入打包器
ADR-04：应用内更新不用 electron-updater
ADR-05：剪藏桥用 HTTP 而不是自定义协议
ADR-06：剪藏桥默认开启
二十、 事故复盘
20.1 v3.1.2 启动闪退（P0）
20.2 v3.1.4 仍打不开（P0）
20.3 依赖地狱（P1，连续三次）
20.4 CI 步骤把自己搞挂（P1）
20.5 设置面板展不开（P2）
二十一、 结语

LongPost v3.2.3 技术架构与产品白皮书

> **文档定位**：面向架构评审、团队交接、技术选型与二次开发者的完整说明。 **覆盖范围**：Web 云端、桌面工作台、Android、Chrome 扩展、本地桥与生态接入。 **数据基准**：仓库 `main` 分支 v3.2.3（提交 `dd84c3f`）。 **配套文档**：产品报告 · 使用说明 · 剪藏 API · 生态接入

目录

章节	内容
一	执行摘要：LongPost 在解决什么问题
二	版本演进：从 2.7 到 3.2
三	系统架构总览
四	三端形态与存储拓扑
五	Web 云端平台（21 模块 / 125 路由）
六	桌面工作台（Electron 渲染层与主进程）
七	双工作区：极简版 / 专业版设计系统
八	内容引擎：Markdown、图表 DSL 与导出矩阵
九	AI 体系：三级取用与密钥托管
十	v3.0 三件套：订阅、分发、迁移
十一	v3.1–3.2 生态层：本地桥、扩展、CLI、Obsidian
十二	安全体系：威胁模型与防护清单
十三	质量工程：测试矩阵与 CI 流水线
十四	性能、构建与产物
十五	部署与运维
十六	数据模型
十七	已知限制与路线图
十八	附录：目录、环境变量、快捷键、API 速查

一、 执行摘要

1.1 长文创作的三重断裂

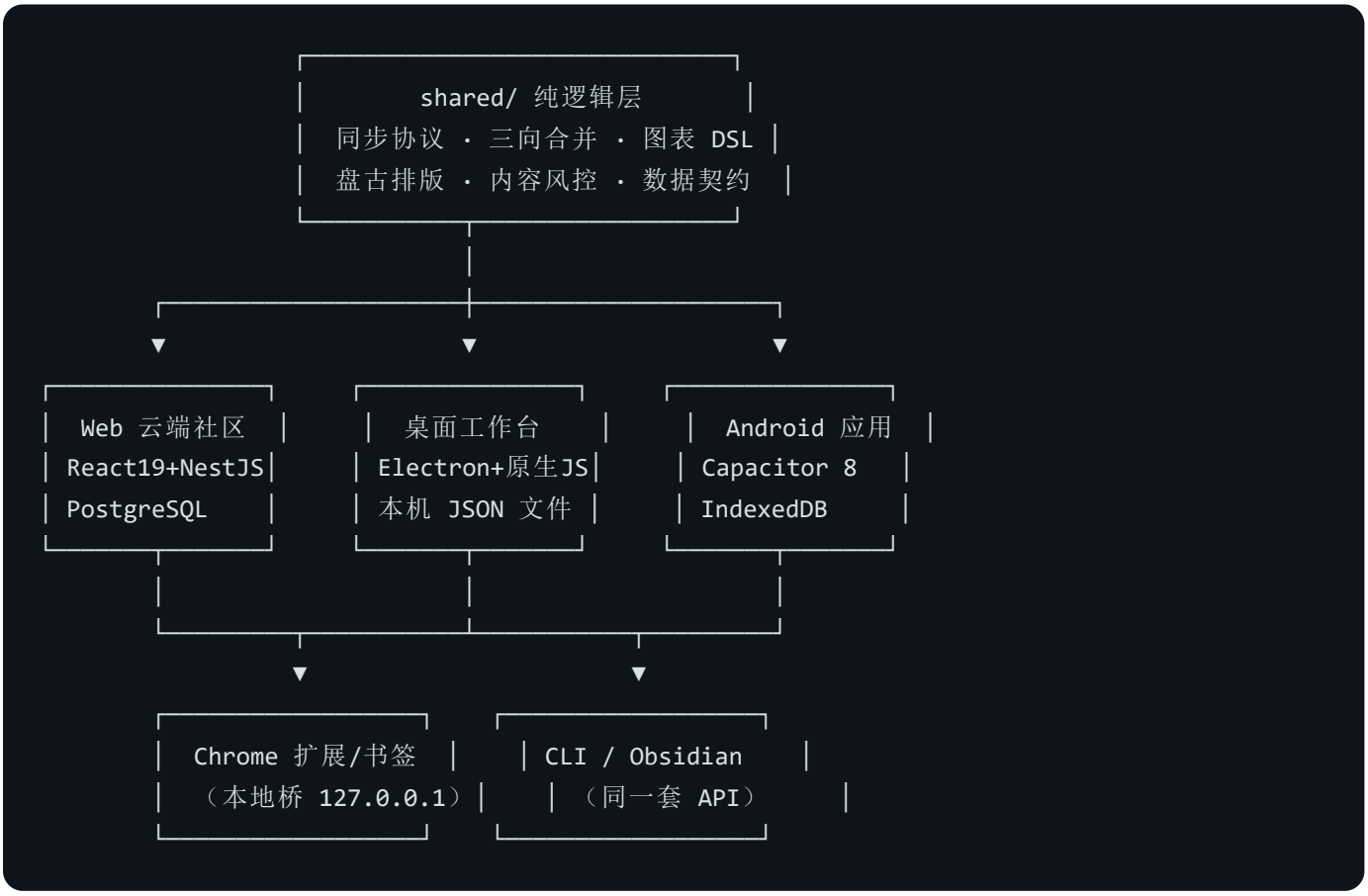
传统内容生态把「写」和「发」拆成了互不相通的两半：

形态	代表	优势	断裂点
中心化内容平台	公众号、知乎、X	有流量、有互动	强制登录、排版受限、数据不属于作者、无法内嵌活数据
本地笔记工具	Obsidian、Notion、Logseq	离线、纯文本、安全	发布链路重、分享体验差、无社交反馈
自建博客	Hexo、Ghost	自主可控	门槛高、无互动、移动端体验差

三者的共同盲区是：**写作者同时需要「离线安全感」和「公网传播力」，但市面上的工具默认你只能二选一。**

1.2 LongPost 的答案

LongPost 用一套代码库同时提供两种形态：



三条设计原则贯穿全项目：

1. **本地优先**：桌面与移动端 100% 离线可用，只有 AI、WebDAV、更新检查会出站。
2. **单一事实源**：算法与协议放 `shared/`，两端引用同一份实现，避免行为漂移。
3. **可验证**：每个功能都有对应的自检或单元测试；改坏了 CI 会拦住。

1.3 关键数字（v3.2.3 实测）

指标	数值
服务端模块	21 个
HTTP 路由	125 条
前端页面	21 个
服务端代码	120 文件 / 18,702 行
前端代码	229 文件 / 45,319 行
桌面渲染层	45 文件 / 22,190 行
共享逻辑	10 文件 / 1,864 行
单元测试	26 文件 / 225 项全通过
桌面自检	78 项
移动自检	108 项
生产依赖	32 个
生产依赖漏洞	0

二、 版本演进

版本	主题	关键变化
v2.7.5	双工作区雏形	布局分叉、杂志排版、崩溃恢复
v2.9.0	生态互通	公众号排版、RSS/Atom、Obsidian 导出、PWA Share Target
v2.9.5	分享修复	微信多场景、QQ 扫码、直达二维码
v2.9.7	工程治理	三套影子目录归档、测试直连生产代码、SSRF 修复、首屏 −86%
v2.9.8	安全与体验	桌面/安卓接入 AI、依赖漏洞 17→0、登录节流、设置面板重构、设计令牌收敛
v2.9.9–3.0.3	界面落地与稳定性	应用内更新、对话框椭圆修复、启动崩溃两连修、锁定官方 npm 源
v3.0.0	个人发布引擎	邮件订阅（双向确认/发布推送/退订）、动态 OG 分享图、Obsidian 导入、整站静态导出
v3.1.0–3.1.9	Chrome 连接	本地桥 + 令牌配对、书签小工具、MV3 扩展、发送到菜单、剪藏 API 文档
v3.2.0–3.2.3	生态与治理	命令行剪藏、Obsidian 插件、依赖体检、README 版本守卫

三个转折点值得单独说：

- **v2.9.7** 之前，测试是“复制一份代码再测”，改完经常忘记同步；之后改成直连生产代码，测试才开始有意义。
- **v3.0** 之前，LongPost 只回答“怎么写”；之后开始回答“写完之后”。
- **v3.1** 之前，入口只有“我坐下来写”；之后变成“我在哪都能开始写”。

三、 系统架构总览

3.1 分层

```
┌──────────────────────────────────────────────────────────────┐
│ 表现层                                                       │
│  client/ (React 19)   desktop/renderer/ (原生 JS)            │
│  chrome-extension/ (MV3)   obsidian-plugin/ (CommonJS)       │
├──────────────────────────────────────────────────────────────┤
│ 接入层                                                       │
│  NestJS REST /api (125 路由)   Socket.IO (私信/会议信令)      │
│  SSE (/api/ai/chat)          本地桥 HTTP (127.0.0.1)         │
├──────────────────────────────────────────────────────────────┤
│ 领域层                                                       │
│  server/modules/* (21 个)   desktop/lib/* (渲染层共享模块)    │
│  shared/* (同步协议、图表 DSL、盘古排版、数据契约)             │
├──────────────────────────────────────────────────────────────┤
│ 基础设施                                                     │
│  Drizzle ORM → PostgreSQL 16    本机 JSON / IndexedDB        │
│  Sharp (图片)  Nodemailer/Resend (邮件)  ECharts/KaTeX/Prism │
└──────────────────────────────────────────────────────────────┘
```

3.2 为什么不做成一套

社区要联网、要鉴权、要实时；写作台要求断网可用、数据 100% 在本机。硬揉成一套，两边都会做不好。所以：

- `client/` 与 `desktop/renderer/` 是**两套独立渲染层**，不共享 UI 代码；
- 它们共享的是**协议与算法**（`shared/` 与 `desktop/lib/`），不是组件；
- Web 与桌面之间的数据交换通过 **导入/导出 + WebDAV 同步**，不靠私有协议。

3.3 一次阅读请求的完整链路

```
浏览器 GET /p/:slug
  → NestJS ViewController（SSR）
  → Drizzle 查询 article + author + images + attachments
  → renderArticleBody()（Markdown → HTML，图表围栏 → chart 容器）
  → buildArticleHtml()（注入 OG 标签、结构化数据、首屏样式）
  → 返回完整 HTML
  → 客户端 hydrate：按需懒加载 ECharts / KaTeX
```

免登录短链是这套系统的关键设计：**访客不注册也能读完一篇带图表的长文。**

四、 三端形态与存储拓扑

端	运行时	数据位置	离线	主要场景
桌面	Electron 44 (Node 22)	`%APPDATA%/LongPost/data/articles.json` + `files/`	100%	深度写作、归档、导出
Android	Capacitor 8 WebView	IndexedDB	100%	碎片记录、移动阅读
Web	Node 22 + React 19	PostgreSQL	需联网（PWA 有缓存）	公开分发、社区互动、会议

4.1 桌面端存储细节

```
%APPDATA%/LongPost/
├── data/
│   ├── articles.json      # 文章正文与元数据
│   ├── settings.json      # 偏好、AI 配置、桥令牌
│   └── files/             # 图片与附件（按 hash 命名）
├── bridge.json            # 本地桥端口与令牌（运行时生成）
└── logs/                  # 运行日志
```

写入策略：**先写临时文件再 rename**，避免中途崩溃留下半个 JSON。

4.2 移动端

Capacitor 只打包 `desktop/renderer/` 一个目录，所以渲染层里不能出现 `../lib/xxx.js` 这种跳出目录的引用 —— 所有共享模块都要通过 `npm run vendor` 复制到 `renderer/vendor/`。

这条约束用一个自检兜底：复制完逐字节比对，不一致直接报错。

五、 Web 云端平台

5.1 模块地图（21 个）

模块	职责	关键点
`auth`	注册、登录、OAuth、JWT、改密	登录失败节流（IP+邮箱，15 分钟 5 次锁 15 分钟）
`posts`	短帖	字数限制、话题解析、转发引用
`articles`	长文 CRUD、发布、系列	发布即触发订阅推送（幂等）
`comments`	评论与楼中楼	软删除、点赞、按时间/热度
`interactions`	点赞、收藏、转发	计数与通知联动
`follows`	关注关系	关注流、粉丝列表
`notifications`	通知中心	已读游标、聚合
`hashtags`	话题	热榜、内容关联
`dm`	私信	Socket.IO 实时、会话列表
`meetings`	视频会议	6 位会议码、WebRTC 信令、纪要
`uploads`	文件上传	后缀白名单、随机文件名、Sharp 重编码
`view`	SSR 阅读页与 Feed	免登录短链、OG 注入、RSS/Atom
`sync`	WebDAV 代理	四层 SSRF 守卫
`ai`	AI 网关	三级取用、密钥加密托管、SSE 流式
`newsletter`	订阅与推送	双向确认、幂等推送、CSV 导出
`og`	动态分享图	SVG → PNG、字体探测
`access-codes`	通行码空间	一码一空间、使用日志
`admin`	后台管理	用户、内容、举报、站点设置
`site-settings`	站点配置	注册模式、开关项

模块	职责	关键点
`pwa`	PWA 清单与图标	可安装、Share Target
`hello`	健康检查	部署探活

5.2 路由分布

125 条 HTTP 路由按前缀分组：

前缀	说明
`/api/auth/*`	登录注册、OAuth 回调、改密、偏好
`/api/posts/*`	短帖时间线、详情、转发
`/api/articles/*`	长文 CRUD、发布、系列
`/api/comments/*`	评论增删查
`/api/interactions/*`	点赞收藏转发
`/api/follows/*`	关注与粉丝
`/api/notifications/*`	通知
`/api/hashtags/*`	话题与热榜
`/api/dm/*`	私信
`/api/meetings/*`	会议
`/api/upload`	上传
`/api/sync/*`	WebDAV 代理
`/api/ai/*`	AI 状态、密钥、对话
`/api/newsletter/*`	订阅、确认、退订、统计、后台
`/api/og/*`	分享图
`/p/:slug` `/a/:slug`	免登录阅读页
`/feed.xml` `/atom.xml`	订阅源

5.3 SSR 与客户端渲染的边界

```
服务端渲染（首屏可见、SEO 友好）
  ├── 阅读页 /p/:slug
  ├── RSS / Atom
  └── 分享图 /api/og/article/:id

客户端渲染（交互、实时）
  ├── 时间线、编辑器、私信、会议
  └── 设置、后台
```

阅读页走 SSR 是因为：爬虫（微信、X、Slack）拿 OG 标签时不执行 JS。

5.4 数据库访问

Drizzle ORM，纯参数化查询。建表不依赖迁移工具，而是**幂等 DDL**：

```
CREATE TABLE IF NOT EXISTS app_users (...);
ALTER TABLE app_users ADD COLUMN IF NOT EXISTS token_version INTEGER NOT NULL DEFAULT 1;
```

好处：新环境 `npm start` 就能建库；坏处：改列类型要手写补丁。当前二十余张表的规模下，这个取舍是划算的。

六、 桌面工作台

6.1 进程结构

```
主进程 (main.js)
├── 窗口与菜单
├── IPC 面：文章读写、文件导入导出、AI 代理、WebDAV、更新、剪藏桥、发送到
├── bridge.js      本地桥 HTTP 服务
├── lib/store.js   仓库层（repo + 磁盘后端）
└── lib/updater.js 应用内更新

渲染进程 (renderer/app.js)
├── 原生 JS，无框架、无打包器
├── 通过 window.LPPlatform 访问能力（平台门面）
└── vendor/ 下是共享模块与第三方库的拷贝
```

6.2 为什么渲染层不用框架

- 桌面与 Android 共用同一份渲染层，Capacitor 只打包一个目录；
- 没有打包器就没有构建步骤，改完直接生效；
- 代价是必须自己管状态与事件，所以代码里到处都是"先查元素再绑定"的防御写法。

6.3 IPC 面（节选）

通道	方向	说明
`articles:list/save/delete`	双向	文章读写
`files:import`	双向	图片附件导入
`ai:chat/abort`	双向	AI 流式对话
`update:check/download/install`	双向	应用内更新
`bridge:info/setEnabled/rotateToken/log`	双向	剪藏桥
`sendTo:install`	双向	写 Windows 发送到快捷方式
`bridge:clipped`	主→渲染	剪藏完成通知

6.4 打包清单守卫

踩过的坑：新增 `desktop/bridge.js` 但忘了加进 electron-builder 的 `build.files`，开发时一切正常，装到用户机器上启动即崩：

```
Error: Cannot find module './bridge'
```

现在有 `scripts/check-package-files.js`：静态解析 main/preload 里的 `require('./x')`，逐个比对 `build.files`，不通过就退出码 1，并且已经挂进 CI。

同类保险还有一层：`main.js` 里所有可选功能的 `require` 都包了 try/catch，**任何一个可选模块缺失都不能让应用打不开。**

七、 双工作区设计系统

7.1 两套骨架，不是两套配色

维度	极简版	专业版
布局	无侧栏、880px 居中	侧边抽屉 + 右侧探索栏
导航	底部悬浮 Dock	顶部工具带 + 浮动主按钮
排版	首字下沉、标点挤压	等宽严谨、结构化
独有工具	沉浸心流	知识图谱、写作实验室、版本差分、白板

切换方式：`Alt + M` 或点击模式按钮，偏好持久化。

7.2 设计令牌

全部收敛到 `:root`：

```
--r-xs 5 / --r-sm 7 / --r-md 10 / --r-lg 14 / --r-xl 20 / --r-pill 999
--fs-xs 11.5 / --fs-sm 12.5 / --fs-md 13.5 / --fs-lg 15 / --fs-xl 17 / --fs-2xl 20
--dur-1 120ms / --dur-2 200ms / --dur-3 320ms
--ease cubic-bezier(0.2, 0.8, 0.2, 1)
```

v2.9.8 做过一次收敛：圆角从 15 种字面值降到 6 个令牌，字号从 19 种降到 6 个，transition 从 24 处裸秒数降到 3 个。

7.3 响应式的真实边界

工具条 16 个按钮塞进 699px 放不下。策略是**两档降级**：

1. **紧凑模式**：隐藏按钮文字只留图标（实测 16 个按钮从溢出变成 646px）；
2. **折叠**：极端窄窗口才按优先级收进「更多」菜单。

手机上只留 3 个高频按钮（置顶 / 查找 / 更多），其余全在菜单里 —— 并且有一条自检断言「折叠掉的按钮在菜单里都有等价入口」。

八、 内容引擎

8.1 Markdown 渲染

`desktop/lib/markdown.js` 是渲染层与导出共用的实现：

- CommonMark 子集 + 表格 + 任务列表 + 高亮 ==text==
- 代码高亮：Prism（20 种语言，离线）
- 数学：KaTeX（`$...$` 与 `$$...$$`）
- 图表：`chart` 围栏 → ECharts 容器

8.2 图表 DSL

```
```chart
{ "type": "bar", "title": "每月发布", "xAxis": ["3月","4月"],
  "series": [{ "name": "发布", "data": [4,7] }] }
```
```

四种类型：bar / line / area / pie。导出的 HTML 里图表仍然可交互（ECharts 打包进单文件）。

8.3 导出矩阵

格式	场景	特点
自包含 HTML	发给别人双击看	图片内联、图表可交互
分享长图 PNG	朋友圈/聊天	2 倍图、按正文全高
公众号排版	粘贴到公众号	内联样式，粘贴即成文
Obsidian MD	进第二大脑	带 frontmatter
标准 MD	备份迁移	纯文本
EPUB	电子书	EPUB 3.0、图片打包
PDF	打印归档	学术排版

8.4 排版算法

`shared/` 里的「盘古」逻辑负责中英文之间自动加空格、标点规范化。它被三个地方引用：编辑器实时预览、导出、公众号排版 —— 所以三处观感一致。

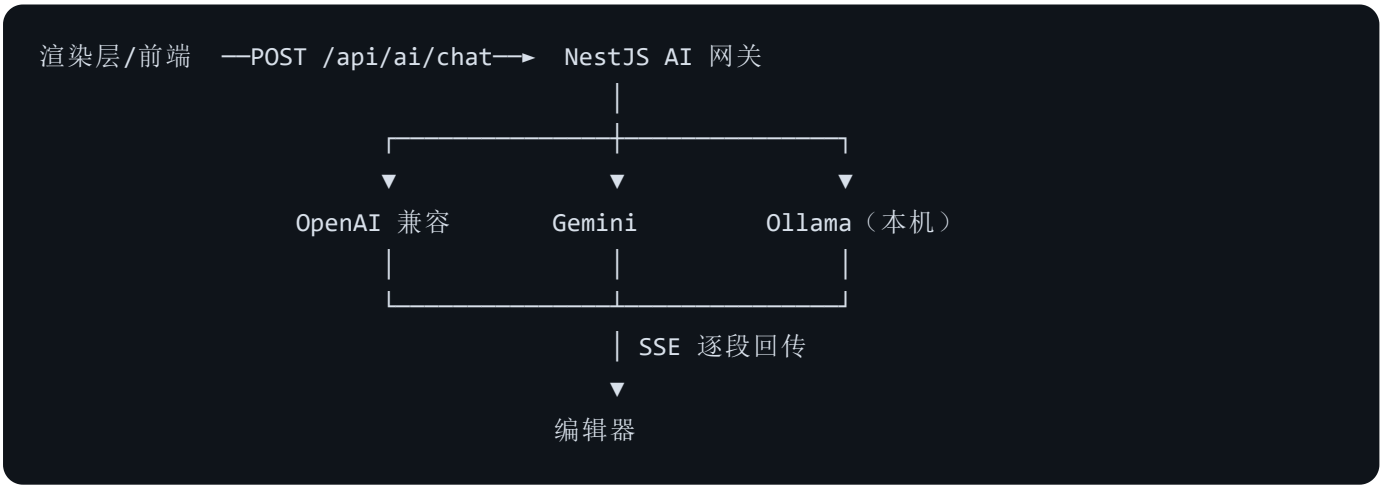
九、 AI 体系

9.1 三级取用

级别	凭据来源	谁付费	适用
① 本机 Key	用户填在设置里，只存本机	用户	桌面/安卓主力
② 账号托管	填一次，AES-256-GCM 加密入库	用户	多设备（仅 Web）
③ 站点额度	站长配置，登录即可用，按天限次	站长	新用户试用（仅 Web）

路由优先级：本机 Key → 托管 Key → 站点额度。

9.2 网关设计



为什么要有网关而不是前端直连模型：

- 托管 Key 与站点 Key **不能下发到浏览器**；
- 需要统一做配额、限流、请求体上限；
- 需要把不同厂商的流式格式归一化成同一种 SSE。

9.3 密钥托管

- 加密：AES-256-GCM，密钥由 `AI_KEY_SECRET`（缺省回退 `JWT_SECRET`）派生；
- 生产环境没配 secret 时**拒绝托管**，而不是用一个公开常量加密；
- 界面只显示掩码，永远不回显明文。

9.4 场景化能力

不是「右侧放个聊天框」，而是嵌进写作动作：

动作	输入	输出
润色	选中文字	更通顺的版本
精简	选中文字	删冗词留主干
排版	选中文字	加空格、规范标点
续写	上文	1–2 段延续
引流短帖	整篇文章	3 个标题 + 1 条短帖 + 1 组 Thread

引流短帖的提示词里做了长度保护：超过 6000 字按「开头 60% + 结尾 40%」截取 —— 结论通常在文章后半段，只截开头会把最该被引用的话丢掉。

十、 v3.0 三件套

10.1 订阅（Newsletter）

```
读者填邮箱 → POST /api/newsletter/subscribe → 发确认信
    → 点确认链接 → status=active
    → 作者发布文章 → 逐封发送 → 幂等锁（article_id 唯一）
    → 退订链接 → status=unsubscribed
```

三个刻意的决定：

1. **订阅接口永远返回 202** —— 不暴露邮箱是否存在，否则它就是探测接口；
2. **双向确认** —— 既符合反垃圾邮件要求，也保护被同事随手填了邮箱的人；

3. **发送失败不抛异常** —— 推送是后台动作，一封失败不该让整批中断。

邮件通道三条：`console`（只打日志）、`resend`（HTTP）、`smtp`（nodemailer）。

10.2 分发（OG 分享图）

- 入口：`GET /api/og/article/:id.png`
- 实现：拼 SVG → sharp 转 PNG（不引入 satori，省掉 10MB 字体）
- 折行按字符宽度：中日韩算 1，ASCII 算 0.55
- **字体是命门**：容器里没有中文字体就会渲染成方块，所以有 `detectCjkFont()` 启动探测
- 文章没有封面图时，SSR 的 `og:image` 直接指向动态图

10.3 迁移（导入 / 导出）

导入：选一个 Markdown 文件夹，递归读取，解析 frontmatter，目录名变标签，同名默认跳过（覆盖是最危险的动作）。

导出：生成可直接托管到 GitHub Pages 的静态站：

```
static-site/
  index.html
  posts/<slug>.html
  feed.xml
  sitemap.xml
```

关键决定：**全部相对链接。**GitHub Pages 的项目站挂在 `/repo-name/` 下，绝对路径一换目录就断。

十一、 v3.1–3.2 生态层

11.1 本地桥协议

```
Chrome 扩展 / 书签 / CLI / Obsidian 插件
        │  POST http://127.0.0.1:<随机端口>/clip
        │  Header: X-LongPost-Token: <48 位十六进制>
        ▼
desktop/bridge.js
        │  令牌恒定时间比较 → 限流 → 体积检查 → 规范化
        ▼
store.saveArticle() → 本地文章库（标签「剪藏」+ 来源链接）
```

约束	实现
只监听回环	`listen(0, '127.0.0.1')`
端口分配	系统随机；撞车重试 3 次
认证	`X-LongPost-Token`，定长比较
限流	60 秒 40 次 → 429
体积	512 KB → 413
跨域	`Allow-Origin: *` + **Private Network Access** 放行
来源校验	只接受 http(s)，`file://` → 400

11.2 一个必须讲清楚的坑：Private Network Access

公网 https 页面访问 `127.0.0.1` 时，Chrome 会先发一个带 `Access-Control-Request-Private-Network: true` 的预检；服务端不回 `Access-Control-Allow-Private-Network: true`，请求在预检阶段就被拦。

表现是「点书签没反应」——不是连不上，是浏览器不让连。

11.3 端口与令牌发现

外部程序（CLI / Obsidian 插件）不要求用户手填配置，而是读：

```
// %APPDATA%/LongPost/bridge.json
{ "port": 52341, "token": "9f2c..." }
```

应用启动时写入，关闭开关时删除。

11.4 Chrome 扩展（MV3）

- 右键菜单：保存整页 / 只存选区
- 弹窗：保存整页、保存选区、从剪贴板保存、最近剪藏
- 选项页：配对码、默认标签
- 正文提取：`article/main/[role=main]` → 打分挑块 → 整页 innerText 兜底
- **canvas 应用检测**：Google Docs 这类页面 DOM 里没有正文，明确提示改用剪贴板

11.5 命令行与 Obsidian

```
node scripts/longpost-cli.cjs --title "灵感" --tag 碎片 "一段内容"
echo "管道内容" | node scripts/longpost-cli.cjs
```

Obsidian 插件读 `bridge.json`，把最近 5 条剪藏写成带 frontmatter 的笔记：

```
---
source: https://example.com/post
captured: 2026-10-19T03:12:00.000Z
---
```

十二、 安全体系

12.1 威胁模型与处置

风险	处置	状态
SSRF（WebDAV 代理）	协议白名单 → 主机黑名单 → DNS 内网拦截 → 连接期 lookup 复核	✅
密码爆破	IP+邮箱计数，15 分钟 5 次锁 15 分钟	✅
JWT 泄露	tokenVersion 即时吊销	✅
托管 Key 泄露	AES-256-GCM 加密入库，界面只显掩码	✅
弱密钥兜底	生产环境未配 secret 时拒绝托管	✅
上传攻击	随机文件名、后缀白名单、**不收 SVG**、Sharp 重编码	✅
富文本注入	导出前清理 `<script>` / `onerror=` / `javascript:`	✅
邮件注入	标题摘要全量 HTML 转义	✅
订阅探测	订阅接口无论存在与否都返回 202	✅
本地桥滥用	回环绑定 + 令牌 + 限流 + 体积上限	✅
依赖漏洞	生产依赖 0 漏洞	✅

12.2 供应链治理

2026-10 的教训：仓库长期用 npmmirror，镜像给过一批**官方源不存在的版本**（colord 3.0.0、strip-dirs 3.0.1、trim-lines 3.0.2、raw-body 3.0.3、mime-types 3.0.3），CI 一装就 404。

现在的治理组合：

1. `.npmrc` 指向官方源；
2. `scripts/check-deps.js`：lockfile 不许有镜像地址 + `npm ci --dry-run` 必须可复现；
3. 两个工作流都在装依赖前跑它。

12.3 可选功能不得拖垮启动

`main.js` 里所有可选模块的 require 都在 try/catch 里；剪藏桥缺失时应用照常启动，只是功能不可用。

这条规则的由来：v3.1.2 因为 `bridge.js` 没打进包，用户装完直接闪退。

十三、 质量工程

13.1 测试矩阵

套件	数量	说明
单元测试 `test/*.test.mjs`	**225**	26 个文件，直连生产代码，不经编译
桌面自检 `smoke-test.js`	**78**	渲染、图表、导出、图标、主进程语法、AI 接线
移动自检 `verify-mobile.js`	**108**	仓库层、Capacitor 配置、触摸目标、许可证、样式回归

单元测试覆盖：同步协议、三向合并、SSRF 守卫、登录节流、AI 加密与配额、LLM 流式解析、更新器、内容风控、CSV 注入、盘古排版、EPUB、图表数据、热榜游标、剪藏规范化、HTTP 协议、配对码、OG 模板、静态站、Markdown 导入、依赖与文档守卫。

13.2 三个「守卫」脚本

脚本	拦截什么
`check-package-files.js`	本地 require 没被打进安装包（启动即崩）
`check-deps.js`	镜像残留、lockfile 不可复现（CI 404）
`check-doc-version.js`	README 版本与 package.json 不一致（文档撒谎）

前两个在桌面工作流里，第三个在两个工作流里，全部在构建前跑。

13.3 CI 流水线

```
推 main / 推 tag
  → 取代码 → 文档版本校验 → 装依赖 → 前端构建
  → 依赖体检 → 打包清单校验 → 逻辑自检 → 打包
  → 生成校验和 → 发布 Release（tag 版为 Latest）
```

13.4 质量门禁

```
npm run type:check      # 双端 TypeScript
npm run eslint          # 0 error
npm test                # 225 / 225
npm run build           # server + client
npm run check:deps      # 依赖体检
npm run check:docs      # 文档版本
cd desktop && npm test && node scripts/verify-mobile.js
```

十四、 性能与产物

指标	数值
服务端编译	120 文件（SWC，约 0.3s）
前端构建	约 2.5s
桌面打包	约 3 分钟（三平台并行）
入口 chunk	约 393 kB（gzip 124 kB）
`dist/client` 总量	约 7.5 MB

优化手段：路由级 `React.lazy`、Mermaid/KaTeX/ECharts 动态导入、`emptyOutDir` 修正。

十五、 部署与运维

15.1 三种方式

方式	适合	命令
Docker Compose	个人/小团队	`docker compose up -d --build`
PM2	已有 Node 环境	`pm2 start ecosystem.config.cjs`
直接跑	调试	`npm run build && npm start`

15.2 必填环境变量

变量	说明
`JWT_SECRET`	不填会用默认值，等于谁都能伪造登录态
`OWNER_EMAIL`	站长邮箱
`DATABASE_URL`	PostgreSQL 连接串
`UPLOAD_DIR`	上传目录，必须是持久化卷

15.3 可选：订阅与 AI

```
NEWSLETTER_ENABLED=true
MAIL_PROVIDER=resend        # 或 smtp / console
RESEND_API_KEY=re_...
SITE_URL=https://你的域名

AI_SITE_PROVIDER=gemini
AI_SITE_KEY=...
AI_DAILY_QUOTA=20
```

15.4 备份

```
docker compose exec db pg_dump -U longpost longpost > backup-$(date +%F).sql
docker run --rm -v longpost_uploads:/data -v "$PWD":/backup alpine tar czf /backup/uploads.tar
```

十六、 数据模型

核心表（节选）：

表	说明
app_users	用户、角色、偏好、tokenVersion
posts	短帖
articles	长文（status: published/draft）
article_images / article_attachments	长文附件
comments	评论（软删除）
interactions	点赞/收藏/转发
follows	关注关系
notifications	通知
hashtags / post_hashtags	话题
dm_conversations / dm_messages	私信
meetings	会议与纪要
access_codes / access_code_logs	通行码空间
user_ai_settings	托管 Key 密文与偏好
newsletter_subscribers	订阅者（pending/active/unsubscribed）
newsletter_issues	推送幂等锁（article_id 唯一）

十七、 已知限制与路线图

17.1 限制

1. 桌面/安卓不能用站点 AI 额度（无账号体系）。
2. 多实例部署需 Socket.IO Redis adapter，PM2 配置锁单实例。
3. 真实 Chrome 内的扩展安装与点击流程，未在自动化环境中验证。
4. 扩展的正文提取打分权重凭经验设定，尚未用真实样本校准。
5. 开发依赖仍有告警（构建工具链，不进运行时）。
6. 无代码签名：macOS 首次打开需右键放行，Windows 可能触发 SmartScreen。

17.2 路线图

优先级	事项	理由
P1	端到端测试（真 Chrome + 真 vault）	生态层目前只有单测
P1	提取算法用真实样本调参	现在无法评估"抓得准不准"
P2	多实例 Redis adapter	水平扩容
P2	代码签名 + 安卓 release keystore	消除安装提示
P3	Obsidian 社区插件上架	需要 PR 审核
P3	Chrome 商店上架（$5）	需要付费账号

十八、 附录

18.1 目录结构

```
longpost/
├── client/            React 19 前端（@ → client/src）
├── server/            NestJS 12 后端（sourceRoot = server）
├── shared/            纯逻辑与类型（无 DOM / 无框架依赖）
├── desktop/           Electron 应用（含 renderer 与 vendor）
├── mobile/            Capacitor 安卓壳（webDir 指向 desktop/renderer）
├── chrome-extension/  MV3 扩展
├── obsidian-plugin/   Obsidian 插件
├── test/              Node 原生测试
├── scripts/           开发、打包、守卫脚本
├── docs/              白皮书与文档
└── archive/           历史原型（不参与构建）
```

18.2 常用命令

命令	作用
`npm run dev`	前后端并行开发
`npm run build`	全量构建
`npm test`	单元测试
`npm run check:deps`	依赖体检
`npm run check:docs`	文档版本校验
`cd desktop && npm run dist:win`	打 Windows 安装包
`cd desktop && node scripts/smoke-test.js`	桌面自检

18.3 快捷键

快捷键	作用
`Ctrl/Cmd + S`	立即保存
`Ctrl/Cmd + B`	加粗
`Ctrl/Cmd + F`	查找替换
`Ctrl/Cmd + K`	命令面板（Web）
`Alt + M`	切换极简版 / 专业版
`Tab / Shift+Tab`	缩进 / 反缩进
`/`	斜杠指令菜单

18.4 剪藏 API 速查

```
POST http://127.0.0.1:<port>/clip
X-LongPost-Token: <token>
Content-Type: application/json

{ "title": "...", "url": "https://...", "content": "...", "tags": "a,b" }
```

状态码	含义
200	已写入
400	请求体不合法 / 非 http(s)
401	令牌错误
413	超过 512 KB
429	60 秒内超过 40 次

> **文档维护约定**：本白皮书与代码同仓，版本号必须与 `package.json` 一致（`npm run check:docs` 会校验 README，白皮书由人工核对）。数字类指标（测试数、路由数、代码行数）应在每次大版本发布前重新统计。

十九、 关键设计决策记录（ADR）

ADR-01：桌面与 Web 不共享 UI 代码

- **背景**：社区要联网、要鉴权、要实时；写作台要断网可用、数据在本机。
- **决策**：`client/` 与 `desktop/renderer/` 各自独立，只共享协议与算法。
- **代价**：同一个功能要做两遍 UI（比如分享面板）。
- **收益**：两边的约束不互相污染，桌面端不会因为 Web 加了个依赖就变重。
- **复核信号**：只在桌面端有意义的功能就不做 Web 版（如「发送到」菜单）。

ADR-02：测试直连生产代码

- **背景**：早期把代码复制一份到测试目录再测，改完经常忘记同步，测试通过但线上是坏的。
- **决策**：`test/*.test.mjs` 直接 import `server/`、`desktop/lib/` 的源文件，Node 24 原生剥离类型。
- **代价**：测试依赖源码结构，重构文件路径要同步改测试。
- **收益**：测试与生产代码不可能漂移。

ADR-03：渲染层不引入打包器

- **背景**：Capacitor 只打包 `desktop/renderer/` 一个目录。
- **决策**：原生 JS + 全局脚本，共享模块通过 `vendor/` 拷贝。
- **代价**：没有模块系统，共享模块必须自己包 IIFE 防全局污染（踩过：两个模块都写 `const API`，第二个直接 SyntaxError）。
- **收益**：改完刷新即生效，没有构建等待。

ADR-04：应用内更新不用 electron-updater

- **背景**：当前分发方式是 CI 出包 + 免安装版，没有代码签名。
- **决策**：自己实现「拉 release 清单 → 下载 → 校验 sha256 → 拉起安装程序」。
- **代价**：没有差分包，每次全量下载。
- **收益**：链路短、可控、能强制校验哈希。

ADR-05：剪藏桥用 HTTP 而不是自定义协议

- **背景**：书签与扩展都要能把内容送进来。
- **决策**：本地 HTTP + 令牌；自定义协议 `lp://` 只用于打开本地文件。
- **理由**：协议跳转拿不到返回结果，用户不知道成没成；HTTP 能直接告诉调用方 200 还是 401。

ADR-06：剪藏桥默认开启

- **背景**：默认关会让「装完不能用」，默认开会让用户对本地端口有顾虑。
- **决策**：v3.1.1 起默认开启，设置里可关闭，关闭时真正停掉监听。
- **复核信号**：若收到杀毒软件误报，再考虑改回默认关。

二十、 事故复盘

20.1 v3.1.2 启动闪退（P0）

现象：安装后双击无反应，任务管理器里也看不到窗口。

根因：新增 `desktop/bridge.js` 未加入 electron-builder 的 `build.files`；打包后 `require('./bridge')` 抛 MODULE_NOT_FOUND，且发生在 `createWindow()` 之前，窗口从未创建。

修复：补 `build.files`；可选模块的 require 全部包 try/catch；新增 `check-package-files.js` 并挂进 CI。

教训：CI 绿只代表「构建成功」，不代表「装上去能跑」。发布前必须真的启动一次打包产物。

20.2 v3.1.4 仍打不开（P0）

现象：修完 bridge 仍然没有窗口。

根因：启动时调用 `ensureClipBridge()`，但该函数定义在另一个作用域，ReferenceError 同样发生在 `createWindow()` 之前。

修复：改为模块级转发函数；启动顺序调整为「先注册 IPC，再起桥」。

教训：主进程是纯 JS，没有类型检查，作用域错误只能靠 `node --check` 加真实启动发现。

20.3 依赖地狱（P1，连续三次）

现象：CI 构建全部失败，报 404。

根因：仓库长期用 npmmirror，镜像给过一批官方源不存在的版本；删 lockfile 也没用，`node_modules/.package-lock.json` 会把旧版本带回来。

修复：`.npmrc` 换官方源；删 `node_modules` 与隐藏 lockfile 后重装；`check-deps.js` 进 CI。

教训：供应链问题不会自己消失，必须有一个自动化的守门人。

20.4 CI 步骤把自己搞挂（P1）

现象：加了「依赖体检」之后，桌面构建反而失败。

根因：步骤插在「装依赖」之前，且继承了 `working-directory: desktop`，找不到仓库根的脚本。

修复：移到安装之后，显式指定仓库根目录。

教训：给流水线加守卫时，要按「守卫自己也会失败」来设计顺序。

20.5 设置面板展不开（P2）

现象：点设置里的分组卡片没反应。

根因：`dialog-body` 是 flex 列容器，卡片默认 `flex-shrink: 1`，展开时被邻居挤压，外层永远不产生滚动。

修复：`.settings-card { flex: none; }`。

教训：视觉 bug 也要用数据确认（量高度、量 scrollHeight），不能只靠肉眼。

二十一、 结语

> **写作要离线，分发要在线，数据要能带走。**

三端分离是为了第一条；SSR + 分享图 + 订阅是为了第二条；导入导出 + 静态站 + WebDAV 是为了第三条。

工程上的坚持只有两条：**能被测试证明的才算做完**，**能被用户装上的才算发布**。